Independent Work 1: Technical Report

Drone Management System with Java Collection Framework

Name: Sheripova Begimai Group: COMSE-25A

About the project

This project is a drone management system with two kinds of drones.

CameraDrone records video.

DeliveryDrone delivers packages.

Both extend the abstract class `Drone`. Their abilities are interfaces: `Chargeable`, `Recordable`, `Deliverable`.

Other classes:

`Pilot`: one pilot has many drones (1:N).

`Request`: a client asks to record a video or deliver a package.

`DroneService`: keeps all collections and all logic.

Roles:

Client puts a request into the queue.

Operator takes requests from the queue with `poll()` and gives them to a drone.

Before, the drones were stored in arrays. Now everything is stored in JCF collections. The old classes and relations stay the same.

UML class diagram



Which collections I used and why

Collection	Where	Why
`Set<Drone>` (HashSet)	all drones	no duplicates
`Map<Integer, Drone>` (HashMap)	drones by id	find a drone fast, O(1)

Collection	Where	Why
`Map<Integer, Pilot>` (HashMap)	pilots by id	find a pilot fast
`Map<String, List<Drone>>`	drones by type	group drones by type
`List<Drone>` (ArrayList)	drones of a pilot	1:N relation
`Queue<Request>` (LinkedList)	requests	first in, first out (FIFO)

Each collection has its own job, so there is no repeated code. I used a simple FIFO queue, because the task allows it and it is fair: the first request is processed first.

equals, hashCode and compareTo

`equals()` and `hashCode()` are in `Drone`, `Pilot` and `Request`. They use `id` as the unique key. Two objects with the same `id` are equal, so `Set` does not accept duplicates and `Map` finds objects correctly.

`Drone` implements `Comparable<Drone>`. `compareTo()` sorts by model name, then by id.

Three comparators sort drones in different ways:

Comparator	Sorts by
`BatteryComparator`	battery level
`FlightTimeComparator`	flight time
`AltitudeComparator`	max altitude (high first), then model name

Relations and roles

1:N: `Pilot` has a `List<Drone>`. `assignDrone()` adds a drone to a pilot.

Client: `submitRequest()` adds a request to the end of the queue.

Operator: `processRequests()` takes requests with `poll()`. It looks for a drone of the right type with battery 20% or more (for delivery, the payload must also be enough). The drone does the job and loses 15% battery. If there is no suitable drone, the request fails.

Service methods

Action	Method
Create	`addDrone()`
Read	`findById()`
Update	`updateBattery()`
Delete	`removeDrone()` (removes the drone from all collections)
Filter	`filterByBattery()`, `getByType()`
Sort	`getSorted()`, `getSorted(comparator)`
Queue	`submitRequest()`, `processRequests()`
Report	`printReport()`

Edge cases that are handled:

`null` or duplicate drone: `IllegalArgumentException`.

Drone or pilot not found: `NoSuchElementException`.

Empty queue: the program prints a message and does not crash.

No suitable drone: the request is counted as failed.

Battery is kept between 0 and 100.

Unknown type: an empty list is returned, not `null`.

Program output

`Main` creates 4 drones, 2 pilots and 4 requests.

Duplicate id and search by id

```
ошибка: дрон с id 1 уже есть

поиск по id 3: дрон-доставщик #3 Cargo (90.0%)
ошибка: дрон с id 99 не найден
```

Sorting and filter

```
по модели (compareTo)
  камера-дрон #2 Air (15.0%)
  дрон-доставщик #3 Cargo (90.0%)
  дрон-доставщик #4 Heavy (50.0%)
  камера-дрон #1 Mavic (80.0%)
по заряду
  камера-дрон #2 Air (15.0%)
  дрон-доставщик #4 Heavy (50.0%)
  камера-дрон #1 Mavic (80.0%)
  дрон-доставщик #3 Cargo (90.0%)
по времени полёта
  камера-дрон #2 Air (15.0%)
  дрон-доставщик #4 Heavy (50.0%)
  дрон-доставщик #3 Cargo (90.0%)
  камера-дрон #1 Mavic (80.0%)
по высоте, потом по модели
  камера-дрон #1 Mavic (80.0%)
  камера-дрон #2 Air (15.0%)
  дрон-доставщик #3 Cargo (90.0%)
  дрон-доставщик #4 Heavy (50.0%)
заряд от 50
  камера-дрон #1 Mavic (80.0%)
  дрон-доставщик #3 Cargo (90.0%)
  дрон-доставщик #4 Heavy (50.0%)
```

Request queue

Request 3 (12 kg) fails, because the delivery drones can carry only 5 kg and 10 kg.

```
клиенты подают заявки
подана заявка #1 от Асель, съёмка 10 мин
подана заявка #2 от Тимур, доставка 4.0 кг
подана заявка #3 от Нурлан, доставка 12.0 кг
подана заявка #4 от Мээрим, съёмка 5 мин

оператор обрабатывает очередь
берём заявка #1 от Асель, съёмка 10 мин
Mavic снимает видео 10 мин в разрешении 20 МП
берём заявка #2 от Тимур, доставка 4.0 кг
Cargo доставляет груз весом 4.0 кг
берём заявка #3 от Нурлан, доставка 12.0 кг
нет подходящего дрона, заявка не выполнена
берём заявка #4 от Мээрим, съёмка 5 мин
Mavic снимает видео 5 мин в разрешении 20 МП
очередь пустая
```

Update, delete and report

```
после удаления дрона 3
  камера-дрон #2 Air (100.0%)
  дрон-доставщик #4 Heavy (50.0%)
  камера-дрон #1 Mavic (50.0%)

отчёт
всего дронов: 3
дрон-доставщик: 1
камера-дрон: 2
средний заряд: 66.66666666666667%
самый заряженный: камера-дрон #2 Air (100.0%)
выполнено заявок: 3
не выполнено заявок: 1
пилот Айгуль, дронов: 1
пилот Бексултан, дронов: 1
```

The system now uses Java Collection Framework: `Set` for unique drones, `Map` for fast search and grouping, `List` for the 1:N relation and `Queue` for requests. `equals`, `hashCode` and `compareTo` use the `id` key. The program also has roles, CRUD, a report and error handling.